Log to Incident

Table of Contents

Chapter 1. Purpose

Large incident resolution times hurt business operations and customer satisfaction. One way to shorten resolution is to bring critical application problems to the right operations team quickly and with enough context to act. That requires bridging **observability** (detecting and describing problems from runtime signals) with **IT service management** (tracking work as incidents, assignment, and closure).

This document defines that bridge as the **Log to Incident** process: selected application log messages become managed incidents for the application operations team. The process is named for the **signal type** and **outcome**, not for any particular vendor or tool. Implementations may use different observability and ITSM platforms; this document's worked example uses Apache Spark as the application, with platform-specific sections for how those platforms are configured.

Chapter 2. Log to Incident

Log to Incident is the Observability process that detects and turns **critical** application log messages into **tracked** IT incidents so the application operations team can respond in time.

2.1. The Process

The Log to Incident process takes one or more log messages and generates an incident in the ITSM platform through a three phase process:

1. The application logs a message
2. The observability tooling detects the message as a being salient and creates an alert and sends it to the ITSM platform
3. The ITSM platform determins which alerts are salient and then creates an incident for operators to track.

Figure 1 illustrates these phases in more detail.

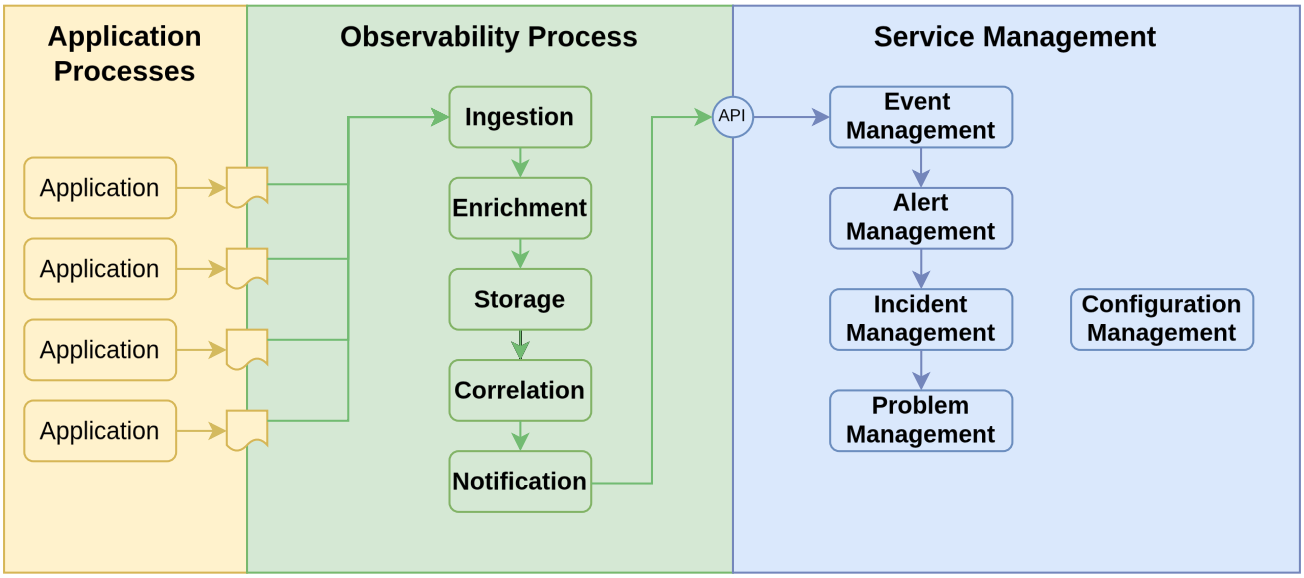

Figure 1. Log to Incident process phases

2.1.1. Phase 1: Application Logging

To various degrees applications are written to generate log messages to a file or other storage medium or APIs. The log messages can indicate forward progress in the application or problems that have occurred. The problems can range from minor inconsistencies to major failures that require immediate attention.

2.1.2. Phase 2: Observability Tooling

The observability platform ingests the log messages either through an agent reading the log files or via an API that the application calls. Either way, the observability platform processes each log message in four addition steps.

1. **Enrichment**: The observability platform adds additional information to the log message. This information comes from the context of the log file (e.g. name of the log file) as well as by parsing information within the log file. This information can include can include the application name, the environment, the host name, the process id, the thread id, the timestamp, the log level, the log message, and the log message id.
2. **Storage**: The observability platform takes the enriched log message and stores it in a database for fast retrieval and futher processing.
3. **Correlation**: The observability platform correlates the log message with other log messages or events to group related log messages into a single message to increase the signal to noise ration and to identify patterns and relationships with other log messages or events.
4. **Notification**: The observability platform then takes selected messages or message groups and`` sends the enriched log message to the ITSM platform as an event.

Coordination between the application developers and the observability platform engineers. Ideally, this collaboration would begin in the application design phase to ensure that the log messages are structured in a way that that simplifies the enrichment process and identification of crticical log messages.

2.1.3. Phase 3: ITSM Platform

The ITSM platform accepts events from the observability platform and determines which events (aka log messages in this cae) are salient and then creates an incident for operators to track. This happens in a three stage process with a fourth follow on stage to handle multiple incidents that are related to the same underlying problem.

1. **Event Management**: Event management accepts an incoming event (log message) from the observability platform. Events may be clear events that indicate a past issue has been resolved. Or they may be events that indicate a new issue has been detected. Salient events are then forwarded to Alert Management.
2. **Alert Management**: Alert Management receives the salient events from Event Management and creates an alert record in the Alert Management platform. Alerts are bound (mapped) to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an alert to a CI requires designing the CMdb model and the information in the event so that Alert Management can use the information sent in the event(s) to map the alert to the correct CI. Multiple events may be mapped to a single alert depending on a message key or other criteria. Alert management will take a clear event will close out an existing, an alert with the same message key. Salient alerts are then forwarded to Incident Management.
3. **Incident Management**: Incident Management is where operations teams interact with incidents to resolve the root causes of the incident. When incident management receives a salient alert, it will either bind the alert to an existing incident or create a new incident. The incident will also be bound to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an incident to a CI requires designing the CMdb model and the information in the alert so that Incident Management can use the information sent in the alert to map the

incident to the correct CI.

Incident Management will also assign (or determine) the support organization that will be responsible for resolving the incident. The incident will be assigned to the application's support organization and will carry enough log context for diagnosis.

4. **Problem Management**: Problem Management focuses on the root cause analysis of recurring incidents (once service has been restored). A problem will typically map onto several closed incidents. This process is outside the scope of this document.

2.2. Client and server applications

Many systems have both **service-side** components (clustered services, pods, long-running servers) and **client-side** components (drivers, desktops, mobile apps, or other unmanaged endpoints). Clients complicate modeling the application in a CMdb as the client-side components are not managed by the CMdb.

- **Service-side**: map from the running workload's identity to the application service that operations supports.
- **Client-side**: map from a durable log-path or naming contract to a logical application service that represents the client population.

Log to Incident must still hand both scenarios, even when the client is not a managed CI in the CMDB.

2.3. Design considerations

Specific observability platforms (e.g. Dynatrace or Elastic Stack) and specific ITSM platforms (e.g. ServiceNow or Remedy) will have specific requirements for implementing the Log to Incident observability process. However, there are the several design considerations that span the different platforms. These include:

- Which log messages are considered salient and should be turned into an incident?
- How to model the (service-side) application in the CMdb?
- How to model clients in the CMdb?
- What information must an event, whether from a client or a service, include to determine the CI to map to an incident?

2.4. Platform implementations

Subsequent chapters describe how to implement the Log to Incident process on specific observability and service management platforms. To design and configure such systems, we need to have a specific application in mind that we will use to illustrate the process. For historical reasons, we will use Apache Spark as the application. Apache Spark is a sophisticated data and analytics platform that runs on Kubernetes and has both client and and service use cases.

Mnformation on Apache Spark is available at Apache Spark Documentation.

Chapter 3. Spark Archiecture

Before we discuss how to observe and manage an application, we need to understand the architecture of the application. Spark is a Kuberenetes application. The Spark application consists of a Spark Master (Driver) and one or more Spark Workers. The Spark Master is the central point of control for the Spark application. The Spark Workers are the nodes that do the actual work of the application. You can scale the Spark Workers horizontally or vertically depending on available resources. In addition, there is a Spark History Server that is used to store the history of the application. The History Server is a built-in observability components that provides debugging and monitoring capabilities for Spark. The executors on the workers run jobs and constitue the heart of the application.

Spark can run in one of two modes: **Client Mode** or **Cluster Mode**. In cluster mode all components run on the Kubernetescluster. In cluster mode, the Spark Driver runs on a Kubernetes pod and communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. The executors on the workers run jobs and constitue the heart of the application. Figure 2 illustrates the cluster mode architecture.

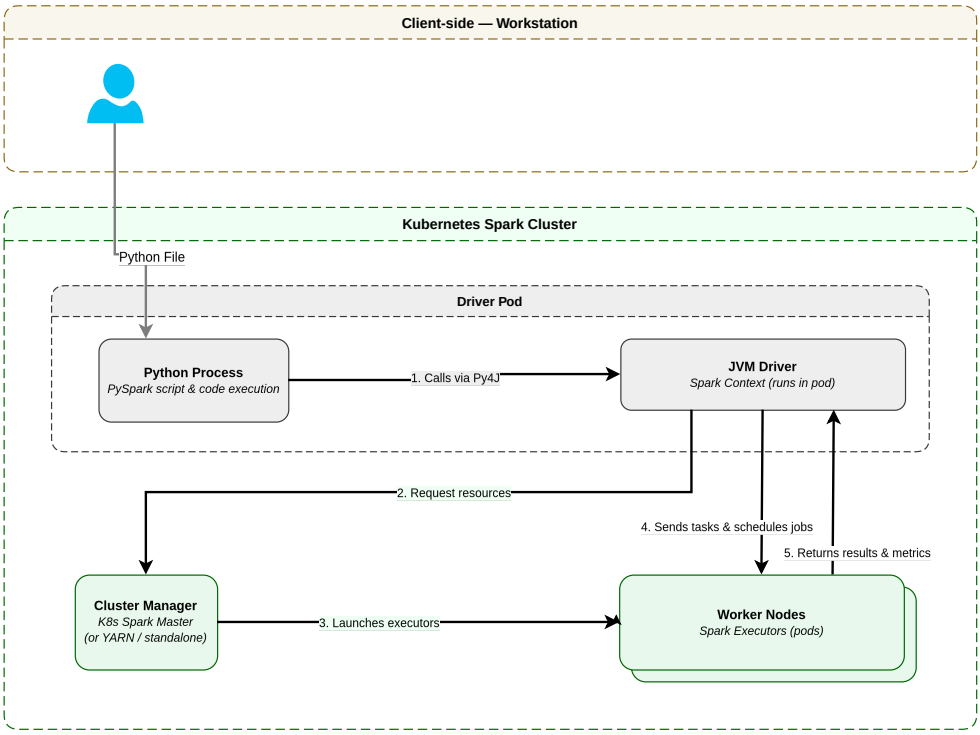

Figure 2. Spark Cluster Mode Architecture

In cluster mode, the users interacts with Spark by submitting a PySpark program to the Spark Driver. This is essentially a batch submission. The Spark Driver then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. The executors on the workers run jobs and constitue the heart of the application.

Client mode is a client-server architecture where the client-side application is a standalone driver that can run on a workstation. Client mode allows data scientists and engineers to work

interatively and iteratively with Spark directly on their own laptops or workstations. client is shown in the [fig-client-mode-architecture].

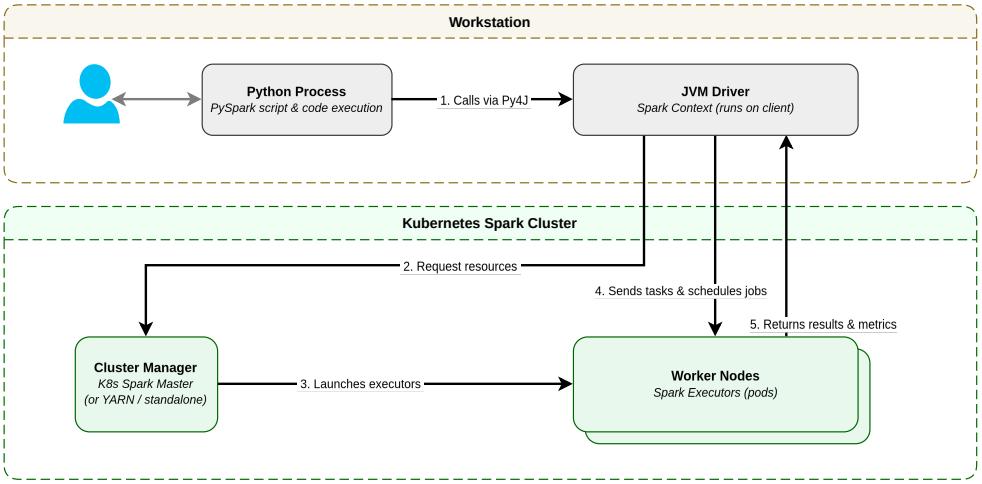

Figure 3. Spark Client Architecture

In client mode, key parts of the Spark Driver run directly on the user's workstation. The user then interacts iteratively with the PySpark process to manipulate Spark data (dataframes and datasets) through PySpark programs and Spark SQL queries. The PySpark process then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side.

Spark inherently supports both Client Mode (client-side) and Cluster Mode (server-side). We have chosen to support both modes within the reference implementation in part because Spark does, but also because, client-side applications occur within many desktop and mobile applications. Client-side applications provide additional challanges in modeling and managing applications.

You can deploy Spark as a client-server application or a server-only application. We had chosen enable the client-server mode to allow data scientists or engineers to launch applications from their own laptops or workstations. This allows for a more flexible and efficient development and testing environment. It also provides a more robust observability excersize as we have to address the client use case, which can occur in many other applicatons. For example, you can see the client in mobile apps where the client is not only a seperate application, but it is an applications that runs on a device that is not managed in ServiceNow's CMDB.

For those not familiar with Spark's client-server model, the Figure 3 illustrates the client archiecture showing the different components of the Spark application in client mode.

Spark can run in one of two modes: **Client Mode** or **Cluster Mode**. In client mode, the client-side application is a standalone application that can run on a workstation or a mobile . In some cases, like this one, the "workstation" may also be a host. In all these cases, the client-side application is not managed by ServiceNow's CMDB. In the case of a true workstation, the workstation is not managed in ServiceNow's CMdb.

The client then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. The executors on the workers run jobs and constitue the heart of the application.

Chapter 4. Dynatrace and ServiceNow

This chapter documents a reference implementation of Log to Incident using **Dynatrace** as the observability platform and **ServiceNow** as the ITSM & Event Management platform used to observe and manage an Apache Spark implementation.

Apache Spark is the sample application (WARN and ERROR log lines escalate in the lab; production agreements often limit escalation to ERROR after review with operations and development).

For more information on Dynatrace see:

- Dynatrace — Log Analytics and OpenPipeline
- Davis problem detection
- ServiceNow — Event Management

4.1. End-to-end flow

The Figure 4, below, illustrates the end-to-end flow log messages from the Spark applicatin through Dynatrace to ServiceNow. This is a for phase process that starts with Spark writing to an application log file; Dynatrace then ingests and processes a log line, deciding which ones are critical issues (Problems) and then notifies ServiceNow of critical issue. ServiceNow ingest the issue (event) and creates an alert and incident mapping bot to appropriate CIs. ServiceNow then notifies the support team of the CI assigned to the incident.

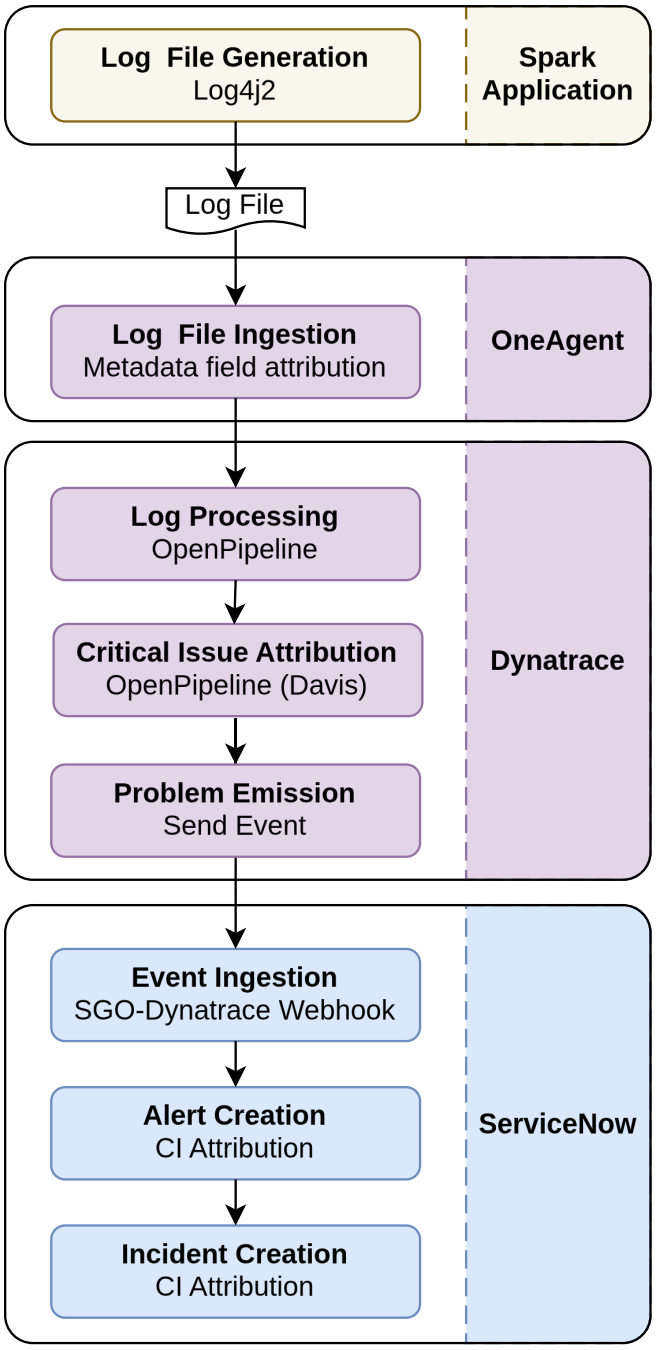

Figure 4. Log to Incident data flow

In ServiceNow, operational issues are managed at the incident level The support group of the CI assigned to an alert or incident is the group that will be notified when an issues arises. If the wrong CI (with an inappropriate support group) is assigned to the incident it will take a longer time to notify the right support team. This makes assigning a CI to an incident critical to the process. For application log messages, the assigned CI should be the application service of the application instance emitting the log message. Finding the right CI at runtime to assign to a log message does not happin automatically. It needs to be designed into the CMdb model of the application.

Designing the CMdb model for the application is another phase in the process. It needs to be done before the application ever runs. The table below enumerates all the major steps required to configure the Log to Incident flow and the touches on some of the key design issues for the step.

Phase	Layer	Design Goals
Phase 1	ServiceNow Modeling	• We need to understand the relationships in the CMdb model and how they relate to runtime dcomponents so that at the time a log message (event) is ingested by ServiceNow, we can eventually assign the correct CI to the incident. This in turns understanding the runtime CIs define by Dynatrace and how they relat to the rest of tghe CMdb model.
Phase 2	Spark Application Configuration	• **Logfile generation:** Where and how does Spark generate the log files and how does Dynatrace's OneAgent extract usefull metadata from them to send to Dynatrace proper?
Phase 3	OneAgent Configuration	• **Log file ingestion:** How does OneAgent ingest the log data and extract usefull metadata from the log files and context and send them to Dynatrace?
Phase 4	Dynatrace Configuration	• **Log Processing:** How does Dynatrace process the data and metadata sent from OneAgent? • **Critical issue determination:** How does Dynatrace determine which log messages are critical and create a Dynatrace problem that should be sent to ServiceNow (as an event)? • **Problem notification:** How does Dynatrace send the Problem (as an event) to ServiceNow?
Phase 5	ServiceNow Configuration	• **Event Ingestion:** How does ServiceNow ingest an event from Dynatrace? • **Alert Creation:** How and when does it create an alert for the event and what CI does it assign to the alert? • **Incident Creation:** When an incident is created, how does ServiceNow identify the correct CI to bind to the incident?

4.2. Phase 1 — ServiceNow Modeling

The first Phase in the process is to model the application in ServiceNow. It needs to happen even before the application ever runs. The ServiceNow modeling is the data foundation of the Log to Incident process. It consists of the CSDM hierarchy and the pattern-specific CMDB components. We need to do this for the Service-side and the Client-side components.

Before we can bind incidents in ServiceNow to a an appropriate CI, we must model the application in ServiceNow, including both the client-side and service-side use cases. Furthermore, we need to model the application in a manner that follows ServiceNow's CSDM best practices. This involves definiing the CSDM hierarchy and disovering pattern-specific CMDB components. We need to do this for the Service-side and the Client-side Spark use cases.

In these models, some CIs (of components) are automatically discovered by ServiceNow and other, typically in the CSDM model, are manually defined to represent the application within an enterprise. Other CIs are imported from other systems, like Dynatrace.

We first examine the CSDM model that is common to both the client and service-side

applications. In the client-side scenario we have both a client and service-side application. As such we examine the service-side model next and then the client-side model after that.

4.2.1. Common CSDM Model

The CSDM is part of the overall ServiceNow model for an application. It models how an application fits within the enterprise. It is a hierarchy of CIs that starts with the application service, which models the application instance. Associated with the application service at a lower (concenpual) level are the CIs that represent the application's Kubernetes components. The Business Application (cmdb_ci_service) represents the business' view of the application - i.e. what more generalized, business oriented, service does it provide. In this case, it is an Apache Spark service that data scientists and engineers use to perform data analytics and machine learning. There could be other instances of the Spark Master service in the enterprise. Lastly, the Business Application represents the overall functional capability area. In this case, it is Data and Analytics. There could be other technologies used in the enterprise aside from Apache Spark (e.g. Snowflake), which would get their own Business Service and still fall under the Data and Analytics Business Application.

Figure 5 illustrates the common CSDM model for the Spark application.

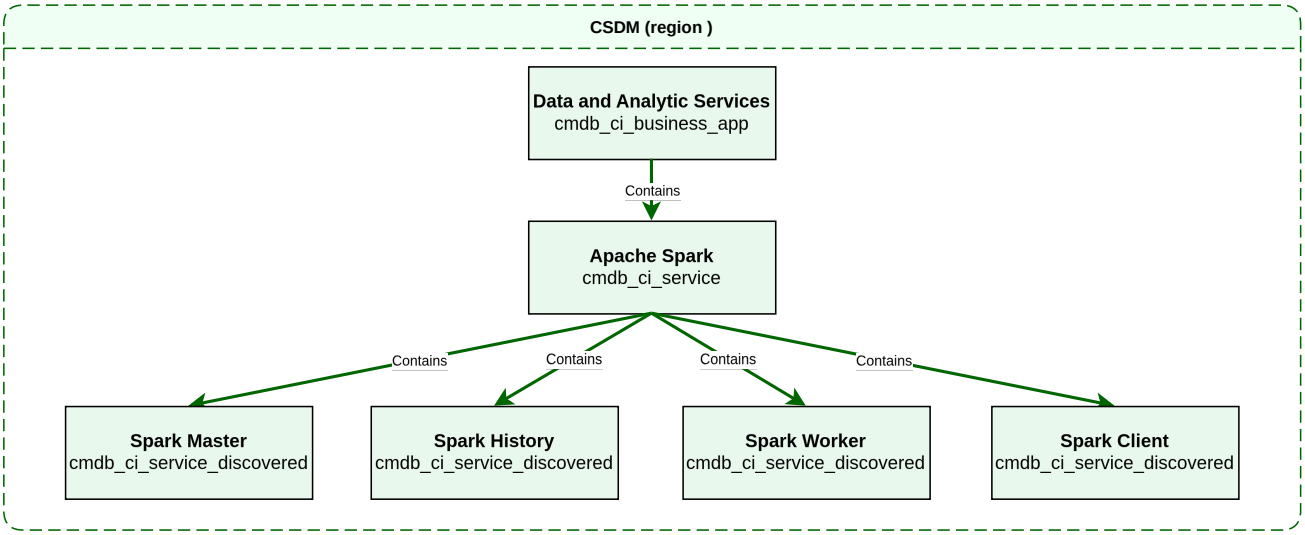

Figure 5. Spark CSDM Model

There is an application service each of the materialized Spark services: the Spark Master, the Spark Workers, and the Spark History Server. The Spark Master is the central point of control for the Spark application. The Spark Workers are the nodes that do the actual work of the application. The Spark History Server is a built-in observability component that provides debugging and monitoring capabilities for Spark. The forth application service is a synthetic application service that represents all Spark clients that talk directly to a the Spark Workers without a using a Spark Master. We will discuss this further in the Figure 7 section.

You must manually create each one of the CSDM CIs in ServiceNow. To simplify the tedium of creating the CSDM CIs, we have used a tool to create the CIs based on a YAML specification. While the tool is beyond the scope of this document we share the YAML specification in Spark CSDM Model for the Spark Master, below. It illustrates most of the key CI attributes needed for the

Spark application.

Spark CSDM Model for the Spark Master

```
business_applications:
  - name: Data and Analytic Services
    identifier: data-and-analytic-services
    short_description: Data processing and analytics platform services
    operational_status: "1"
    active: "true"
    business_owner: gbrooks
    it_application_owner: gbrooks

business_services:
  - name: Apache Spark
    identifier: apache-spark
    short_description: Distributed data processing cluster
    operational_status: "1"
    parent_business_application: Data and Analytic Services
    owned_by: gbrooks
    business_criticality: "4 - not critical"

application_services:
  - name: Spark Master
    identifier: spark-master
    short_description: Spark master — Kubernetes StatefulSet
    operational_status: "1"
    parent_business_service: Apache Spark
    platform: kubernetes
    service_mapping: tags
    discover: false
    environment: on-prem
    location: brooks-lab
    owned_by: gbrooks
    business_criticality: "4 - not critical"
    depends_on:
      - OpenTelemetry Collector
      - Logstash
      - Elasticsearch
      - name: lab3
        type: nfs_server
```

4.2.2. Service-Side Model

The Service-side model used in Spark cluster mode, shows the logical relationships between CSDM Application Services, Kubernetes-discovered CIs, SGC-imported entities, and Event Management bindings for pod-scoped (server side)logs. One of the main outcomes of this model is to be able to map service-side alerts to a reasonable CI and incidents to an application service CI for more timely comunnication to the appropriate support team.

Figure 6 illustrates the Service-side model of the Spark Master. The other Spark services, e.g. Spark Workers and the Spark History Server, would have avery similar CMdb models.

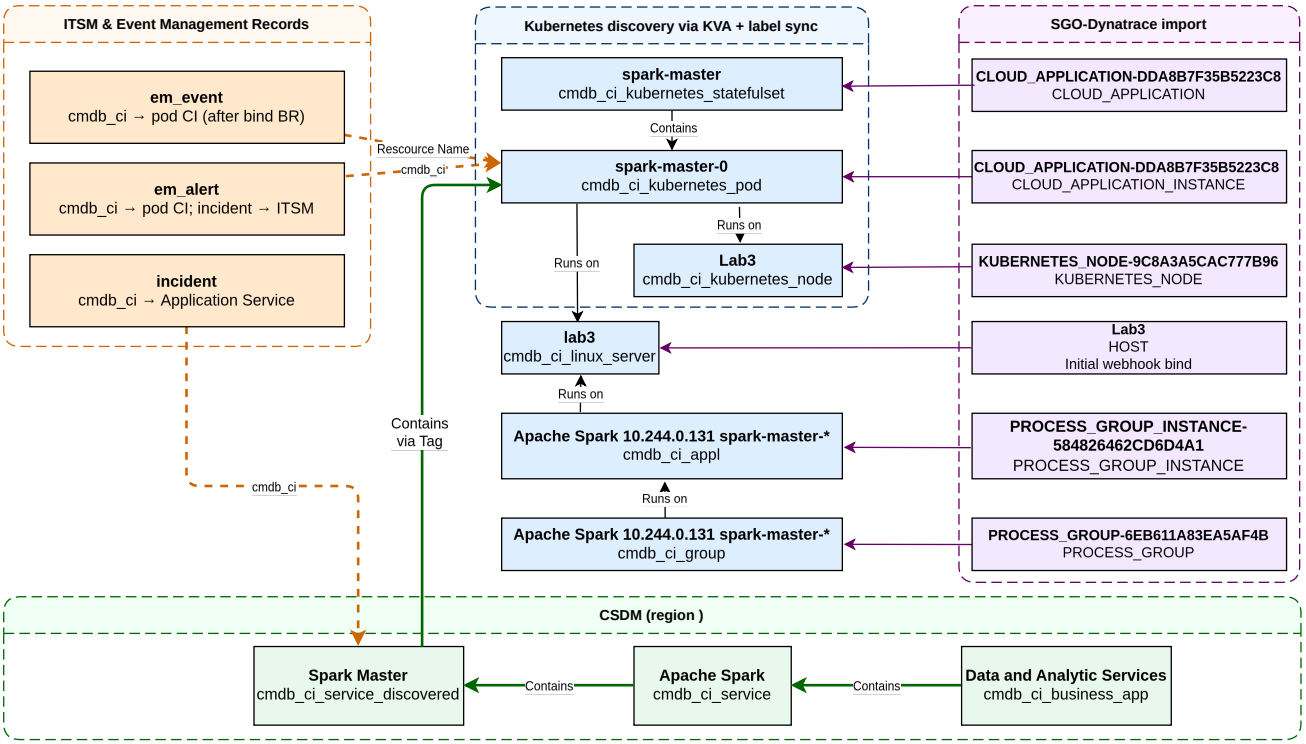

Figure 6. Spark Service-Side Model

The table below enumerates the different CMDB objects in the Service-side scenario. For those CIs that Dynatrace can generate or that you can correlate to a CI, the corresponding Dynatrace entity type is shown. The last column summarizes how Service Graph Connector (SGC / **SGO-Dynatrace**) correlates the Dynatrace entity to the CMDB CI. Key in this process is what the Identity and Reconcilliation Engine (IRE) does to merge the Dynatrace entity with the CMDB CI.

CI Name	CI Class	DT Entity	SGC correlation to CI
spark-master	cmdb_ci_kubernetes_statefulset	CLOUD_APPLICATION	SGC Kubernetes workload import maps `CLOUD_APPLICATION` → workload CI; `entityId` stored in `sys_object_source` (`name=SGO-Dynatrace`). **IRE merges with the KVA StatefulSet when names match.**
spark-master-0	cmdb_ci_kubernetes_pod	CLOUD_APPLICATION_INSTANCE	**SGO-Dynatrace Kubernetes Pod** feed maps `CLOUD_APPLICATION_INSTANCE` → `cmdb_ci_kubernetes_pod`; `entityId` → `sys_object_source.id`. **IRE merges with the KVA pod when `name` matches.**
lab3	cmdb_ci_kubernetes_node	KUBERNETES_NODE	**SGO-Dynatrace Kubernetes Node** feed maps `KUBERNETES_NODE` → `cmdb_ci_kubernetes_node`; `entityId` → `sys_object_source.id`. Merges with the KVA node CI on node name.

CI Name	CI Class	DT Entity	SGC correlation to CI
lab3	cmdb_ci_linux_server	HOST	**SGO-Dynatrace Hosts** feed maps `HOST` → computer / `cmdb_ci_linux_server`; `entityId` → `sys_object_source.id`. **IRE merges with Horizontal Discovery on a case insensitive match of hostname or FQDN.**
Apache Spark 10.244.0.131 spark -master-* - PROCESS _GROUP -6EB611A83EA5AF4B	cmdb_ci_group	PROCESS_GROUP	**SGO-Dynatrace Process Groups** feed maps `PROCESS_GROUP` → `cmdb_ci_group`; `entityId` → `sys_object_source.id`.
Apache Spark 10.244.0.131 spark -master-* spark -master-* spark -master@lab3	cmdb_ci_appl	PROCESS_GROUP _INSTANCE	**SGO-Dynatrace** process / PGI cascade maps `PROCESS_GROUP_INSTANCE` → `cmdb_ci _appl`; `entityId` → `sys_object_source.id`.

To be able to correlate a critical log message to the appropriate application service, we need to use tag-based service mapping to correlate the Kubernetes pod to the application service. This correlation is accomplished by setting the servicenow.io/application-service-identifier attribute in the template for the Kubernetes pod. This attribute is then used to correlate the pod to the application service.

Spark Master StatefulSet pod template for the Spark Master is shown below.

Spark Master StatefulSet pod template labels

```
template:
  metadata:
    labels:
      app: spark-master
      app.kubernetes.io/name: spark-master
      app.kubernetes.io/instance: brooks-lab
      app.kubernetes.io/component: master
      app.kubernetes.io/part-of: apache-spark
      app.kubernetes.io/managed-by: ansible
      servicenow.io/application-service-identifier: spark-master
      servicenow.io/environment: on-prem
      servicenow.io/location: brooks-lab
```

The `servicenow.io/application-service-identifier` attribute is used to correlate the pod to the application service. Each time Kubernetes creates such a pod, the KVA (Kubernetes Visibility Agent) Informer notifies ServiceNow to create a new pod CI. The Informer also sets the `servicenow.io/application-service-identifier` attribute on the pod CI. This attribute is then used to correlate the pod to the application service.

4.2.3. Client-side Model

The client-side scenario is of interest on its own as there are so many workstation and mobile client applications. In the client-side scenario the log message occurs on workstation or mobile

device that is not managed in the CMdb. As ServiceNow does not manage the client device in the CMdb, there are no CIs to attach to any log alert or incident. One way to address this is to create a synthetic CI that represents the application running on any client device. For this we choose the Application Service class(cmdb_ci_service_discovered) to instantiate a CI for the Spark Client. This CI will not have any infrastructure CIs associated with it as the client device is not managed in the CMdb.

Figure 7 illustrates the Client-side model of the Spark Client. It shows a device (and host) independent model consistting of a single Application Service CI that represents the Spark Client.

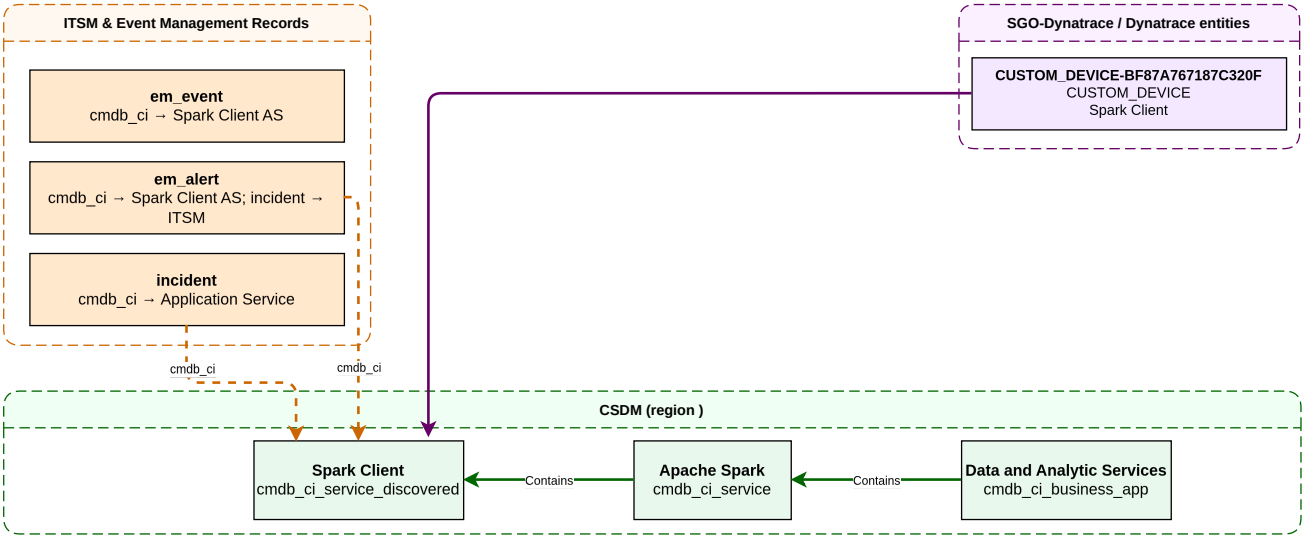

Figure 7. Spark Client-Side Model

The application service YAML specification for the Spark Client is shown below.

Spark CSDM Model for the Spark Client

```
- name: Spark Client
  identifier: spark-client
  platform: host
  service_mapping: manual
  discover: false
  depends_on:
    - Spark Master
```

Now, given the Spark Client application service how do we map a log message on the client device to the Spark Client application service? This is done by creating a custom device entity in Dynatrace that represents the Spark Client. This custome device entity must then be included in any log problems sent as events to ServiceNow from Dynatrace's OneClient agent on the device to Dynatrace's OpenPipeline.

4.2.4. Kubernetes Pod CIs

In this section we drill down into the details of how Kubernetes Pod CIs are created and how they are correlated to the application service. The pod tags are labels that are set on the pod manifest. These labels are then used to create the Kubernetes Pod CIs. Pod manifest labels are the single source of truth for tag-based Service Mapping on Spark workloads.

Tag-based Service Mapping matches workload CIs to an Application Service using `cmdb_key_value` rows copied from these pod labels. The ServiceNow labels have the formation "servicenow.io/<key>". There is no hard standard on how these keys are used. In this implementation, we keys listed in the table below.

Key	Role
`application-service-identifier`	Primary tag filter. Value equals the Application Service `identifier` (for example `spark-master`). Tag-based SM uses this to **Contains** matching pods under that Application Service.
`environment`	Scope filter (for example `on-prem`). Narrows membership so the same application-service identifier in another environment does not match.
`location`	Scope filter (for example `brooks-lab`). Narrows membership across sites.

Of the tag keys listed in the table above, the `application-service-identifier` and `environment` are required. The `location` is optional. The application-service-identifier is used to correlate a pod to the appropriate application service by matching the `identifier` attribute of the Application Service CI. This is a slightly brittle convention that the teams need to keep in mind.

Spark Worker Pod Template illustrates the pod template for the Spark Worker.

Spark Worker Pod Template

```
apiVersion: v1
kind: Pod
metadata:
  name: spark-worker-lab1-0
  labels:
    servicenow.io/application-service-identifier: spark-worker
    servicenow.io/environment: on-prem
    servicenow.io/location: brooks-lab
    app.kubernetes.io/name: spark-worker
spec:
  containers:
    - name: spark-worker
      env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
      volumeMounts:
        - name: app-logs
          mountPath: /opt/spark/logs
          subPathExpr: $(POD_NAME)
  volumes:
    - name: app-logs
      persistentVolumeClaim:
        claimName: spark-logs-pvc
```

4.2.5. ServiceNow — SGC Topology Import

SGC scheduled import	Dynatrace entity type	CMDB class	Role in log incidents
Hosts	HOST	cmdb_ci_linux_server	Fallback when log tails on host and pod entity unavailable

SGC scheduled import	Dynatrace entity type	CMDB class	Role in log incidents
Kubernetes Pod	CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod	Preferred alert/event CI for pod-scoped logs
Process Groups	PROCESS_GROUP	cmdb_ci_appl	Alternative when Davis binds to JVM process
Application Relationships	Smartscape edges	cmdb_rel_ci	Optional topology context

Each imported entity receives a **sys_object_source** row: name=SGO-Dynatrace, id=<Dynatrace entityId>, target_sys_id → CMDB CI sys_id.

Configure on the ServiceNow instance:

- Service Graph Connector for Observability – Dynatrace (scoped app sn_dynatrace_integ)
- Inbound event listener: POST /api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
- Scheduled imports for hosts, Kubernetes pods, and process groups covering the cluster namespace

4.2.6. Dynatrace — management zone

Place hosts, Kubernetes cluster entities, and pods that run the application in a **management zone** dedicated to the integration (for example **Application Observability**). The **alerting profile** for ServiceNow problem notifications must scope to that management zone so only in-scope problems are forwarded.

4.3. Phase 2 — Log File Generation

Now that we have defined the CMdb model for the Spark application, we can walk through the process and configurations needed to emit a log line with requisite metadata.

To start withSpark Log File Sample illustrates a sample log file from the Spark Worker.

Spark Log File Sample

```
2026-07-20 12:40:19 WARN  Utils:250 - Your hostname, Lab3, resolves to a loopback address: 127.0.1.1; using
192.168.1.207 instead (on interface enp3s0)
2026-07-20 12:40:19 WARN  Utils:244 - Set SPARK_LOCAL_IP if you need to bind to another address
2026-07-20 12:40:27 WARN  TaskSetManager:250 - Lost task 0.0 in stage 0.0 (TID 0) (10.244.2.145 executor 0):
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXIS
T] Encountered error while reading file file:///mnt/spark/data/elements/Periodic_Table_Of_Elements.csv. File does not
exist. It is possible the underlying files have been up
dated.
2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job
```

As typical with many applications, there is not a lot of structure to the log files. There is the ever present time stamp, an error level, the file and line number of the log message, and the message itself. Of particular importance is the log level. Instead of matching a long list of critical log

message patterns, we can use it to decide if the log message should be considered a critical issue. For the reference implementation we choose to use both the WARN and ERROR levels. We use warnings for convience as they are easier to generate.

There are several key questions we need to answer:

- Where will we keep the log files?
- What key metadata will we need to extract from the log files?
- What other key metadata will we need about the log files?

4.3.1. Log File Directory Structure

The best practice is to have Dynatrace access the log files directly from the Kubernetes pod. This is the preferred method as it is the most direct and efficient way to get the log files to Dynatrace. However, for legacy reasons, we use a directory-based approach inherited from an Elastic Stack based approach. This is the next best approach as it uses OneAgent to tail the per pod log files from each Kubernetes host.

Spark application log directories on a Kubernetes node (current hostPath approach) shows the **current** layout on a Kubernetes **node** (hostPath `/mnt/spark/logs`, backed by node-local disk — not NFS). Each directory name equals the pod `metadata.name`.Spark application log directories on a Kubernetes node (current hostPath approach) is not a real directory tree as it mixes pod directories from different hosts. For example, the Spark Master and History Server do not share a host with any worker nodes.

Spark application log directories on a Kubernetes node (current hostPath approach)

```
/mnt/spark/logs/                          # node-local hostPath (Lab1Lab3)
├── spark-master-0/                          # StatefulSet ordinal
├── spark-history-server-0/                  # StatefulSet ordinal
├── spark-worker-lab1-7d4f9b8c9d-xk2lm/   # Deployment pod name
├── spark-worker-lab1-7d4f9b8c9d-p4q9n/
└── spark-worker-lab2-5c8a1e2f3b-m7h2k/
```

For Client Mode logs, where the Spark driver is not run in a Kubernetes pod, we use a synthetic path segment `spark-client` under `/mnt/spark/logs/` so client directories are never confused with service-side pod names. Under that segment, **instance** (`SPARK_DRIVER_INSTANCE` or the wrapper PID) separates concurrent drivers on the same host. Logs are node-local, so hostname is not needed in the path. Spark client-mode log directories illustrates the log directory structure for client-mode logs.

Spark client-mode log directories

```
/mnt/spark/logs/                        # node-local hostPath (Lab1Lab3)
├── spark-client/                          # synthetic pod name
    ├── 12345                              # Regular run uses default PID
    ├── par-a                              # Overridden PID for parallel run
    └── par-a                              # Overridden PID for parallel run
```

4.3.2. Log Metadata

The log metadata is the key metadata fields that we need to extract from the log files or the log file context. This depends on what fields are required, the available log file context, the available fields in a log line, Dynatrace's abillity to parse the log file context, and the OpenPipeline's abillity to parse the log file context.

The key requirements are that we extact the log level and a pod identifier. The log level is used to decide if the log message should be considered a critical issue. The pod identifier is used to correlate the log message to the appropriate pod and then in turn to the appropriate application service.

4.3.3. Log4j configuration

In order to pair the OneClient log file readers with the managed (rotated) log files that Spark writes, we need to configure the Log4j appender to write to the correct directory. Here there need to be two different configurations, one for the service-side logs and one for the client-side logs. We use the standard rolling file appender configuration for both.

Service-side (`spark/conf/log4j2-cluster.properties`): RollingFile only. Pods set `SPARK_LOG_DIR` to the pod log directory (on the node: `/mnt/spark/logs/<pod>/`).

Log4j RollingFile — service-side

```
appender.rolling.type = RollingFile
appender.rolling.name = RollingFile
appender.rolling.fileName = ${env:SPARK_LOG_DIR:-/opt/spark/logs}/spark-app.log
appender.rolling.filePattern = ${env:SPARK_LOG_DIR:-/opt/spark/logs}/spark-app-%d{yyyy-MM-dd}-%i.log
appender.rolling.layout.type = PatternLayout
appender.rolling.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex
appender.rolling.policies.type = Policies
appender.rolling.policies.time.type = TimeBasedTriggeringPolicy
appender.rolling.policies.time.interval = 1
appender.rolling.policies.size.type = SizeBasedTriggeringPolicy
appender.rolling.policies.size.size = 20MB
appender.rolling.strategy.type = DefaultRolloverStrategy
appender.rolling.strategy.max = 10
```

Client-side (`spark/conf/log4j2-client.properties`): console + RollingFile. `run-chapters.sh` sets `SPARK_LOG_DIR=/mnt/spark/logs/spark-client/<instance>/` and `-Dlog4j2.configurationFile=⋯/log4j2-client.properties`.

Log4j RollingFile — client-side

```
appender.rolling.type = RollingFile
appender.rolling.name = RollingFile
appender.rolling.fileName = ${env:SPARK_LOG_DIR:-/mnt/spark/logs/spark-client/default}/spark-app.log
appender.rolling.filePattern = ${env:SPARK_LOG_DIR:-/mnt/spark/logs/spark-client/default}/spark-app-%d{yyyy-MM-dd}-
%i.log
appender.rolling.filePermissions = rw-r--r--
appender.rolling.layout.type = PatternLayout
appender.rolling.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex
appender.rolling.policies.type = Policies
appender.rolling.policies.time.type = TimeBasedTriggeringPolicy
appender.rolling.policies.time.interval = 1
appender.rolling.policies.size.type = SizeBasedTriggeringPolicy
```

```
appender.rolling.policies.size.size = 20MB
appender.rolling.strategy.type = DefaultRolloverStrategy
appender.rolling.strategy.max = 10
```

4.3.4. Log emission Summary

Every tailed Spark log line arrives in Dynatrace as a Grail log record. OneAgent supplies severity and path context; custom log source enrichment (and OpenPipeline fallbacks) add the `spark.*` workload fields used for Davis matching and ServiceNow correlation.

Attribute	How it is set	Value
`content`	OneAgent (raw line)	Full Log4j line
`loglevel`	OneAgent (severity from line text)	`WARN`, `ERROR`, `INFO`, …
`status`	OneAgent (from `loglevel`)	Coarse bucket (`WARN`, `ERROR`, …)
`log.source`	OneAgent	Registered path or path pattern
`spark.log.path`	Custom log source `${0}`	Literal file path on the node
`spark.application`	Custom log source / OpenPipeline	`spark-client` **or** pod name (e.g. `spark-master-0`)
`spark.client.instance`	Custom log source `${1}` (client paths only)	Driver instance directory (`par-a`, PID, …); unset for pods
`k8s.pod_name`	Custom log source `${1}` (service paths only)	Same as `spark.application` for pods; kept for CMDB pod-name matching

4.4. Phase 3 — Dynatrace detects ERROR and WARN log lines

Logs ingested through **OpenPipeline** are evaluated by pipeline processors. Classic builtin:logmonitoring.log-events rules do **not** apply to logs routed through OpenPipeline.

4.4.1. OpenPipeline processor flow

Figure 8. OpenPipeline log alerts processor chain

4.4.2. Dynatrace objects to configure

Object	Settings schema	Purpose
Custom log source	builtin:logmonitoring.custom-log-source-settings	OneAgent tails application log paths on hosts or nodes
Logs routing	builtin:openpipeline.logs.routing	Route matched logs into the log-alerts pipeline

Object	Settings schema	Purpose
Log alerts pipeline	builtin:openpipeline.logs.pipelines	Extract pod name, set dt.source_entity, Davis processor
Alerting profile	builtin:alerting.profile	Forward in-scope problems to ServiceNow notification
Problem notification	builtin:problem.notifications	POST webhook on problem OPEN and RESOLVED

4.4.3. Custom log source

Register path patterns that cover per-workload directories and rotated files.

Key fields: `path` patterns **must** align with Log4j `fileName` / `filePattern` (Log4j RollingFile — service-side; client-side) and OpenPipeline `log.source` matchers (OpenPipeline specification). List **more specific** patterns (`spark-client//`) **before** generic `/mnt/spark/logs//`. Enrichment **must** set:

- `spark.log.path` (`${0}`) — literal file path (do **not** reuse `k8s.*` names for non-Kubernetes paths)
- `spark.application` — `spark-client` for client-mode, or the pod directory name for service-side
- `spark.client.instance` (`${1}`) on client paths; `k8s.pod_name` (`${1}`) on service-side paths (equals `cmdb_ci_kubernetes_pod.name`)

Custom log source (brooks-lab Spark)

```
{
  "schemaId": "builtin:logmonitoring.custom-log-source-settings",
  "scope": "environment",
  "value": {
    "enabled": true,
    "config-item-title": "Spark Lab - application log files",
    "custom-log-source": {
      "type": "LOG_PATH_PATTERN",
      "values-and-enrichment": [
        {
          "path": "/mnt/spark/logs/spark-client/*/spark-app.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.application", "value": "spark-client" },
            { "type": "attribute", "key": "spark.client.instance", "value": "${1}" }
          ]
        },
        {
          "path": "/mnt/spark/logs/spark-client/*/spark-app-*.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.application", "value": "spark-client" },
            { "type": "attribute", "key": "spark.client.instance", "value": "${1}" }
          ]
        },
        {
          "path": "/mnt/spark/logs/*/spark-app.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.application", "value": "${1}" },
            { "type": "attribute", "key": "k8s.pod_name", "value": "${1}" }
```

```
          ]
        },
        {
          "path": "/mnt/spark/logs/*/spark-app-*.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.application", "value": "${1}" },
            { "type": "attribute", "key": "k8s.pod_name", "value": "${1}" }
          ]
        },
        { "path": "/opt/spark/logs/spark-app.log*" }
      ]
    }
  }
}
```

4.4.4. Logs routing

Route OneAgent-sourced records whose log.source matches the application path into the alerts pipeline:

```
{
  "schemaId": "builtin:openpipeline.logs.routing",
  "scope": "environment",
  "value": {
    "routing": [
      {
        "enabled": true,
        "description": "Application ERROR/WARN log alerts",
        "matcher": "matchesValue(log.source, \"/var/log/myapp/*\")",
        "pipelines": ["application-log-alerts"]
      }
    ]
  }
}
```

4.4.5. Log alerts pipeline (complete reference)

The pipeline has three processor stages before storage:

1. **Fields extract** — parse `k8s.pod_name` from `log.source` when the path is literal.
2. **Fields add / entity lookup** — set `dt.source_entity` via `lookupEntity(CLOUD_APPLICATION_INSTANCE, k8s.pod_name)`.
3. **Davis processor** — match **WARN**/**ERROR**; set `event.name` to include pod name; emit `event.description` with full log line.
4. **Bucket assignment** — store in **default_logs** bucket.

Key fields: DQL `matcher` on `log.source`; `lookupEntity`; Davis `event.name`, `event.description`, `dt.source_entity`.

Specification 2.5.4-1. OpenPipeline log alerts pipeline (brooks-lab Spark)

```
{
  "schemaId": "builtin:openpipeline.logs.pipelines",
  "scope": "environment",
  "value": {
```

```
    "displayName": "Spark Lab - log alerts",
    "customId": "spark-lab-log-alerts",
    "processing": {
      "processors": [
        {
          "id": "extract-spark-pod-from-path",
          "type": "dql",
          "matcher": "matchesValue(log.source, \"/mnt/spark/logs/*\") OR matchesValue(log.source, \"
/opt/spark/logs/*\")",
          "dql": {
            "script": "parse log.source, \"LD '/mnt/spark/logs/' LD:k8s.pod_name LD '/spark-app.log'\""
          }
        },
        {
          "id": "resolve-spark-pod-entity",
          "type": "fieldsAdd",
          "matcher": "isNotNull(k8s.pod_name)",
          "fieldsAdd": {
            "fields": [
              {
                "name": "dt.source_entity",
                "value": "lookupEntity(CLOUD_APPLICATION_INSTANCE, k8s.pod_name)"
              }
            ]
          }
        }
      ]
    },
    "davis": {
      "processors": [
        {
          "matcher": "(loglevel == \"WARN\" OR loglevel == \"ERROR\") AND (matchesValue(log.source,
\"/mnt/spark/logs/*\") OR matchesValue(log.source, \"/opt/spark/logs/*\"))",
          "davis": {
            "properties": [
              { "key": "event.type", "value": "ERROR_EVENT" },
              { "key": "event.name", "value": "Spark log {loglevel} on {k8s.pod_name}" },
              { "key": "event.description", "value": "Spark {loglevel} on {log.source}: {content}" },
              { "key": "dt.source_entity", "value": "{dt.source_entity}" }
            ]
          }
        }
      ]
    }
  }
}
```

OpenPipeline **Add fields** (`fieldsAdd`) accepts `lookupEntity(CLOUD_APPLICATION_INSTANCE, <pod-name-field>)` in the API `value` property (Settings API uses `name`, not `field`, for the target attribute). DQL processors in the Processing stage do **not** enable `fetch` or `smartscapeNodes`; do not use Grail notebook lookup syntax there. When lookup misses, `dt.source_entity` remains the OneAgent HOST. The required outcome is `dt.source_entity = CLOUD_APPLICATION_INSTANCE-⋯` before the Davis processor runs.

4.4.6. Davis processor properties

Property	Value
Matcher	(loglevel == "WARN" OR loglevel == "ERROR") AND matchesValue(log.source, "/var/log/myapp/*")

Property	Value
event.type	ERROR_EVENT
event.name	Spark log {loglevel} on {k8s.pod_name}
event.description	Application {loglevel} on {log.source}: {content}
dt.source_entity	CLOUD_APPLICATION_INSTANCE entity when pod resolved; else pass-through process or host
dt.davis.event_timeout	15m (adjust to problem correlation needs)

4.4.7. Example Grail log record at Davis input

Attribute	Example
content	2026-06-27 14:53:46 ERROR Worker:264 - All masters are unresponsive! Giving up.
loglevel	ERROR
log.source	/var/log/myapp/myapp-worker-7d4f9b-xk2lm/application.log
workload.pod_name	myapp-worker-7d4f9b-xk2lm (after fields extract)
dt.source_entity	CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890
host.name	k8s-node-01 (tail host — not the alert CI when pod entity is set)

4.4.8. Alerting profile

Scope the profile to the integration management zone and include ERROR and CUSTOM_ALERT severity levels (Davis log events map to ERROR):

```
{
  "schemaId": "builtin:alerting.profile",
  "scope": "environment",
  "value": {
    "name": "Application Observability - ServiceNow",
    "severityRules": [
      { "severityLevel": "AVAILABILITY", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "ERRORS", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "PERFORMANCE", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "RESOURCE_CONTENTION", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "CUSTOM_ALERT", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" }
    ],
    "eventFilters": [],
    "managementZone": "<management-zone-object-id>"
  }
}
```

4.4.9. Host CPU metric event (optional companion path)

Host CPU problems are **not Spark-specific** unless scoped. Use a **neutral event title** and an **entity filter** so the detector evaluates only hosts in the integration boundary (management zone or named hosts). Different business units may use different thresholds — create **separate metric events** per scope rather than one tenant-wide rule.

```
{
  "schemaId": "builtin:anomaly-detection.metric-events",
  "scope": "environment",
  "value": {
    "enabled": true,
    "summary": "Spark Lab - Host CPU above 80%",
    "queryDefinition": {
      "type": "METRIC_KEY",
      "metricKey": "builtin:host.cpu.usage",
      "aggregation": "AVG",
      "entityFilter": {
        "conditions": [
          {
            "type": "HOST_GROUP_NAME",
            "operator": "EQUALS",
            "value": "<integration-host-group>"
          }
        ]
      },
      "dimensionFilter": []
    },
    "eventEntityDimensionKey": "dt.entity.host",
    "modelProperties": {
      "type": "STATIC_THRESHOLD",
      "threshold": 80.0,
      "alertOnNoData": false,
      "alertCondition": "ABOVE",
      "violatingSamples": 1,
      "samples": 3,
      "dealertingSamples": 3
    },
    "eventTemplate": {
      "title": "Host ({dims:dt.entity.host}) CPU above 80%",
      "description": "Host CPU usage in management zone <name> exceeded 80%.",
      "eventType": "CUSTOM_ALERT",
      "davisMerge": false,
      "metadata": []
    }
  }
}
```

Scope the detector with an **entity filter**: **HOST_GROUP_NAME** (when OneAgent assigns a host group), **MANAGEMENT_ZONE** (numeric zone id), or individual **HOST_NAME** values. Multiple conditions in one filter use **AND** logic — use a single host group or zone rather than listing every host unless each host has its own metric event.

When the integration uses a **management zone**, a **MANAGEMENT_ZONE** filter requires the zone's **numeric id** (not the Settings API objectId). Template: `observability/dynatrace/tenants/<tenant>/integrations/spark-cpu-metric-event.json.j2`.

4.4.10. ServiceNow (Phase 2)

No direct ServiceNow configuration. Ensure SGC scheduled imports cover CLOUD_APPLICATION_INSTANCE and PROCESS_GROUP entity types that Davis may select as dt.source_entity.

4.5. Phase 4 — Dynatrace sends the problem to ServiceNow

4.5.1. Webhook endpoint

```
POST https://<instance>.service-now.com/api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
Authorization: Basic <base64(user:password)>
Content-Type: application/json
```

Use **ProblemDetailsJSON** (v1) in the payload template for SGC — not ProblemDetailsJSONv2 (legacy templates may return HTTP 500 on some instances).

4.5.2. Problem notification payload template

The template in Problem notification payload specification is substituted by Dynatrace at send time. `ProblemDetailsJSON.rankedEvents[]` carries the OpenPipeline Davis properties from OpenPipeline specification (`event.description`, entity ids, severity). Those ranked events are the **problem specification** that flows into ServiceNow `em_event.description` (via `ProblemDetailsText`) and into `em_event.additional_info`.

Specification 2.6.2-1. Problem notification payload (SGC)

```
{
  "ConnectionId": "<sgc-connection-sys-id>",
  "ProblemID": "{ProblemID}",
  "ProblemTitle": "{ProblemTitle}",
  "ProblemSeverity": "{ProblemSeverity}",
  "ProblemImpact": "{ProblemImpact}",
  "ProblemDetailsJSON": {ProblemDetailsJSON},
  "ProblemDetailsText": "{ProblemDetailsText}",
  "ImpactedEntities": {ImpactedEntities},
  "ImpactedEntity": "{ImpactedEntity}",
  "ProblemURL": "{ProblemURL}",
  "State": "{State}",
  "Tags": "{Tags}",
  "PID": "{PID}"
}
```

Enable **notifyClosedProblems** so RESOLVED posts update the same em_event via message_key.

4.5.3. Webhook field mapping

Webhook field	Dynatrace source	ServiceNow target
ProblemID	Stable problem identifier	em_event.message_key (OPEN and RESOLVED deduplication)
ProblemTitle	event.name / Davis title	em_event.message; contributes to description
ProblemSeverity	ERROR, CUSTOM_ALERT, etc.	em_event.severity (numeric mapping per SGC rules)

Webhook field	Dynatrace source	ServiceNow target
ImpactedEntities[].entityId	CLOUD_APPLICATION_INSTANCE-..., PROCESS_GROUP-..., HOST-...	sys_object_source lookup → em_event.cmdb_ci
ProblemDetailsJSON.rankedEvents[]	Ranked events with entityId, eventType	Fallback CI binding parse path
ProblemDetailsText	Davis narrative including event.description	em_event.description (primary operator-facing text)
Tags	Problem tags (Project, Environment, ...)	em_event.additional_info; not CSDM join keys
State	OPEN or RESOLVED	Create vs close/update event
ProblemURL	Deep link to Dynatrace problem	em_event.additional_info

4.5.4. Complete example webhook body

After Dynatrace substitutes placeholders for an ERROR log problem:


```
{
  "ConnectionId": "a1b2c3d4e5f678901234567890abcdef0",
  "ProblemID": "-9876543210987654321",
  "ProblemTitle": "Application log ERROR",
  "ProblemSeverity": "ERROR",
  "ProblemImpact": "INFRASTRUCTURE",
  "ProblemDetailsJSON": {
    "displayName": "Application log ERROR",
    "problemId": "-9876543210987654321",
    "rankedEvents": [
      {
        "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
        "entityName": "myapp-worker-7d4f9b-xk2lm",
        "eventType": "ERROR_EVENT",
        "severityLevel": "ERROR",
        "title": "Application log ERROR",
        "eventDescription": "Application ERROR on /var/log/myapp/myapp-worker-7d4f9b-xk2lm/application.log: 2026-06-
27 14:53:46 ERROR Worker:264 - All masters are unresponsive! Giving up."
      }
    ],
    "startTime": 1719502426000,
    "endTime": -1
  },
  "ProblemDetailsText": "Application ERROR on /var/log/myapp/myapp-worker-7d4f9b-xk2lm/application.log:\n2026-06-27
14:53:46 ERROR Worker:264 - All masters are unresponsive! Giving up.\n\n1 impacted entity: myapp-worker-7d4f9b-
xk2lm",
  "ImpactedEntities": [
    {
      "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
      "name": "myapp-worker-7d4f9b-xk2lm",
      "type": "CLOUD_APPLICATION_INSTANCE"
    }
  ],
  "ImpactedEntity": "myapp-worker-7d4f9b-xk2lm",
  "ProblemURL": "https://<tenant>.apps.dynatrace.com/ui/problems/<problem-id>",
  "State": "OPEN",
  "Tags": "Environment:production,Project:myapp",
  "PID": "-9876543210987654321"
```

```
}
```

4.5.5. Ensuring log fields reach ServiceNow

Layer	Configuration
OpenPipeline Davis processor	Set `event.description` to include `{content}`, `{loglevel}`, and `{log.source}` (Phase 2); set `event.name` to include `{k8s.pod_name}` for per-pod correlation (OpenPipeline specification)
Dynatrace problem notification	When OpenPipeline Davis properties are present, Dynatrace expands them into `ProblemDetailsJSON.rankedEvents[]` (each ranked event includes `eventDescription`, `entityId`, `title`). Dynatrace then assembles `ProblemDetailsText` from those ranked events (including `event.description` text) before the webhook is sent. See Problem notification payload specification.
SGC field mapping (sa_event_field_mapping in sn_dynatrace_integ)	Map `ProblemDetailsText` → `em_event.description`; map `ImpactedEntity` or parsed pod name → `em_event.resource`
em_event business rule (Spark pod bind)	When path or `node` encodes pod name, set `cmdb_ci`, `resource`, `message_key` prefix `K8sLog-{pod}-`, and `message` `Event Log WARN` or `Event Log ERROR`
em_event.additional_info	Persist full `ProblemDetailsJSON` for Event Management rules and incident scripts
em_alert.description	Event Management alert rule copies `em_event.description`; `cmdb_ci` copied from event

Kubernetes labels (servicenow.io/application-service-identifier) are **not** in the webhook payload. They remain on CMDB pod CIs for Phase 5 incident correlation.

4.5.6. ServiceNow (Phase 4)

The **Observability engineer** (ServiceNow administrator with Event Management and SGC access) must verify the push connector — not the monitored application developer:

1. Confirm the Store app **Service Graph Connector for Observability – Dynatrace** (`sn_dynatrace_integ`) is installed and active.
2. In **Event Management** → **Integrations** → **Push Connectors** (or **Connections** → **Observability** → **Dynatrace** in newer releases), locate the connector whose inbound URL ends with:

   ```
   POST /api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
   ```

3. Verify **Basic authentication** credentials match the Dynatrace problem notification profile (brooks-lab: provisioned via Ansible `ensure_sn_dt_webhook_auth.yml`).
4. In the scoped app, open **Event field mappings** (`sa_event_field_mapping` / `$sa_event_map`) and confirm: **ProblemDetailsText** → **description**; **ProblemTitle** → **message**; **ProblemID** →

message_key; severity mapping for ERROR and WARN.

5. Ensure EM **event rules** mark inbound SGO-Dynatrace events **Ready** when `cmdb_ci` is populated (see Event Management rules).
6. Ensure EM **alert rules** create alerts from Ready events and copy `cmdb_ci`, `description`, `resource`, and `node` from the event.

No **ServiceNow** configuration is required on the monitored application itself. No **Dynatrace** changes are required at this Phase beyond the problem notification profile deployed in Phase 2 — Dynatrace.

4.6. Phase 5 — ServiceNow receives the event and binds infrastructure CI

4.6.1. Event listener

Setting	Value
Source	SGO-Dynatrace (URL query parameter on inbound_event)
Scoped app	sn_dynatrace_integ
CI lookup	sys_object_source where name=SGO-Dynatrace and id=<entityId from ImpactedEntities>

4.6.2. em_event field mapping

em_event field	Population
source	SGO-Dynatrace (from URL)
message_key	ProblemID
description	ProblemDetailsText — must include full log line from OpenPipeline event.description
message	ProblemTitle
severity	Mapped from ProblemSeverity (ERROR → 2 Major typical; WARN → 3 Minor if applicable)
type	ERROR or derived from ProblemDetailsJSON ranked event type
node	ImpactedEntity display name (pod name when CLOUD_APPLICATION_INSTANCE)
resource	Log file path or workload.pod_name parsed from ProblemDetailsText or additional_info
cmdb_ci	sys_object_source.target_sys_id for entityId in ImpactedEntities
cmdb_ci_type	cmdb_ci_kubernetes_pod when entity is CLOUD_APPLICATION_INSTANCE; cmdb_ci_appl for PROCESS_GROUP; cmdb_ci_linux_server for HOST fallback

em_event field	Population
additional_info	JSON: ProblemURL, Tags, ImpactedEntities, full ProblemDetailsJSON, PID
time_of_event	Problem start time from ProblemDetailsJSON
correlation_id	PID when mapped

4.6.3. CI binding procedure

1. Listener receives POST at inbound_event?source=SGO-Dynatrace.
2. Parser extracts entityId from ImpactedEntities[0] or ProblemDetailsJSON.rankedEvents[0].
3. Query sys_object_source: name=SGO-Dynatrace, id=<entityId>.
4. Set em_event.cmdb_ci and em_event.cmdb_ci_type from the matched row.

4.6.4. Entity type to CMDB class

Dynatrace entityId prefix	CMDB class	When to expect
CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod	OpenPipeline resolved pod entity (preferred for K8s log paths)
PROCESS_GROUP	cmdb_ci_appl	Log tied to JVM process when pod lookup unavailable
HOST	cmdb_ci_linux_server	Last resort — file tailed on node without pod enrichment

Prerequisites for pod binding:

- SGC Kubernetes Pod import includes pods in the application namespace.
- Pod name in Dynatrace matches Kubernetes metadata.name.
- IRE merge aligns KVA-discovered pod CIs with SGC-imported entities when both exist.
- sys_object_source row exists before the webhook arrives (account for import schedule lag).

4.6.5. Description and alert text

Field	Required content
em_event.description	Line 1: log level and message body from content; Line 2: log.source path; optional ProblemURL appended
em_alert.description	Same as em_event.description (copied by alert rule or inherited on alert create)
em_alert.resource	Pod name or full log file path
em_alert.cmdb_ci	Same infrastructure CI as em_event.cmdb_ci

4.6.6. SGC and Event Management configuration

Component	Specification
sa_event_field_mapping	ProblemDetailsText → description; ProblemTitle → message; ProblemID → message_key; severity mapping for ERROR and WARN
EM event rule	Mark events Ready when severity and cmdb_ci are populated; optional script normalizes description from additional_info if connector mapping is insufficient
EM alert rule	Create alert from Ready events; copy cmdb_ci and description from event; require cmdb_ci for incident automation
ACL on cmdb_key_value	Roles that run incident scripts must read tags on pod CIs (see install documentation for tag ACL requirements)

4.6.7. Application and Dynatrace (Phase 5)

At Phase 5 the **Observability engineer** typically performs **verification**, not new Dynatrace or application configuration:

1. After a test problem fires, confirm `em_event.source=SGO-Dynatrace` and `em_event.cmdb_ci` is populated (pod CI when OpenPipeline entity lookup succeeded).
2. If `cmdb_ci` is a Linux server instead of a pod, return to Phase 3 — Dynatrace and verify `k8s.pod_name` extraction and `lookupEntity(CLOUD_APPLICATION_INSTANCE, k8s.pod_name)` in the OpenPipeline pipeline (OpenPipeline specification).
3. Confirm `sys_object_source` contains a row with `name=SGO-Dynatrace` and `id=<entityId>` from the webhook **before** the event arrives (SGC scheduled import lag is a common root cause of empty CI).
4. Deploy or verify the **K8s log pod bind** business rules (K8s log pod bind business rules) as a fallback when the webhook still reports a HOST entity but the log path encodes a pod name.

No ServiceNow-side changes are required on the monitored **application** (the Java/Spark/Python workload). **No additional Dynatrace** configuration is required at Phase 5 if Steps Phase 3 — Dynatrace and Phase 3 are correct.

ServiceNow cannot bind to `cmdb_ci_kubernetes_pod` unless Phase 3 — Dynatrace sets `dt.source_entity` to the pod entity and Phase 0 imported that entity into `sys_object_source`.

4.7. Phase 6 — Create incident bound to Application Service

4.7.1. Severity policy

Environment	Automatic incident when
Non-production / test	ERROR and WARN (simplifies validation when ERROR signals are hard to trigger)
Production	ERROR only; WARN remains at alert level or creates a lower-priority task per ITSM policy

Incidents must set **incident.cmdb_ci** to **cmdb_ci_service_discovered** (Application Service), not the infrastructure CI on the alert.

4.7.2. Layer model

Layer	Table	cmdb_ci class
Event	em_event	Infrastructure: cmdb_ci_kubernetes_pod, cmdb_ci_appl, or cmdb_ci_linux_server
Alert	em_alert	Same infrastructure CI as the source event
Incident	incident	cmdb_ci_service_discovered (Application Service)

4.7.3. Incident correlation

Incident correlation defines how enriched alerts map to `incident.cmdb_ci`. The business rule `em-alert-create-k8s-log-incident` tries the **Client-side** log path first, then the **Service-side** Contains traversal.

The comparison table summarizes alert CI versus incident CI for each pattern. Full algorithms follow in the adjacent subsections.

Pattern	Alert CI	incident.cmdb_ci resolution
Service-side	`cmdb_ci_kubernetes_pod` (preferred) or HOST fallback	Spark Service-Side Log Pattern — Contains traversal from pod CI
Client-side	HOST or empty (acceptable)	Spark Client-Side Log Pattern — `/logs/spark-client/` → Spark Client AS

4.7.3.1. Service-side

The **Spark Service-Side Log Pattern** resolves Application Services by traversing **Contains::Contained by** from the alert's pod CI. A materialized edge from `cmdb_ci_service_discovered` (parent) to `cmdb_ci_kubernetes_pod` (child) **must** exist before the problem fires.

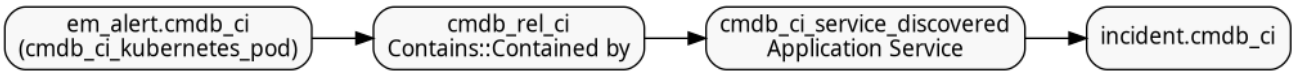

Figure 9. Incident correlation — Spark Service-Side Log Pattern

1. `K8sLogPodCiBind` sets `em_event.cmdb_ci` and `em_alert.cmdb_ci` to the **pod CI** whose `name` matches

the log path segment (Phase 5).

2. Query `cmdb_rel_ci` where `child` = pod CI sys_id and `type` = **Contains::Contained by**.
3. Parent row **must** be `cmdb_ci_service_discovered`.
4. Set `incident.cmdb_ci` = parent Application Service sys_id.

Out of scope: host CI binding without pod rebind, process group CI, tag-only lookup without a Contains edge.

4.7.3.2. Client-side

The **Spark Client-Side Log Pattern** maps log paths to a single logical Application Service. Client-mode driver logs **do not** correspond to a `cmdb_ci_kubernetes_pod`; incident automation **must not** depend on alert CI or Contains edges.

Figure 10. Incident correlation — Spark Client-Side Log Pattern

1. Scan `em_alert.description`, `resource`, and `node` for the substring `/logs/spark-client/`.
2. Look up Application Service `Spark Client` by `name` (see CSDM identifier lookup).
3. Set `incident.cmdb_ci` to that Application Service sys_id.
4. `K8sLogPodCiBind` skips path segment `spark-client` when parsing pod names.

4.7.4. CSDM identifier lookup

CSDM `identifier` in `*.csdm.yaml` is the logical machine key embedded in log paths and optional Dynatrace tags. The table below shows which CMDB fields are reliable for incident lookup on optimizincdemo1.

Purpose	Mechanism	Queryable field on `cmdb_ci_service_discovered`
Manual / logical AS documentation	CSDM `identifier: spark-client` in log path contract	`identifier` column — often **not populated** on optimizincdemo1
Display / incident lookup	CSDM `name` (e.g. `Spark Client`)	`name` — reliable for `resolveFromSparkClientLogPath()`
Optional process tag	`DT_TAGS=servicenow.io/application-service-identifier=spark-client`	Process-group problems only; **not** required for log-path incidents

Spark Client AS **must** have `service_status=operational` (promoted by `csdm/deploy.yml` → `ensure_host_as_operational.yml`).

```
cd ansible
ansible-playbook -i inventory.yml playbooks/servicenow/incident/verify_spark_client_as.yml \
  -e @../vars/secrets.yaml
```

4.7.5. Known limitation — Davis problem bundling

Dynatrace Davis **bundles** multiple log-derived `ERROR_EVENT` rows into **one problem** when they share the same `dt.source_entity` and merging is allowed. Historically both client-mode and Service-side file tails sat on the OneAgent **HOST**, so concurrent WARN/ERROR lines produced a **single** `em_alert.description` with both client and master paths.

Remediation (lab):

- Custom log source / OpenPipeline set `spark.application` (`spark-client` vs pod name) and `spark.log.path` (literal path — not `k8s.log.path`).
- Client events bind `dt.source_entity` to CUSTOM_DEVICE **Spark Client** (keeps them off HOST).
- Client Davis processor sets `dt.davis.is_merging_allowed=false` and `event.unique_identifier=spark-client-{spark.client.instance}` so concurrent drivers do **not** merge into one problem (ServiceNow still maps all client paths to Application Service **Spark Client**).
- Service-side Davis processor likewise sets `dt.davis.is_merging_allowed=false` and `event.unique_identifier={spark.application}` so different pods do not merge into one problem.
- ServiceNow `ResolveApplicationService.applySparkClientAlertBinding()` binds alert CI to **Spark Client** when `/logs/spark-client/` is present.

Worker→multi-client cascading failures are a different topology case; this pipeline does not try to correlate those. It only stops direct client□client and client□HOST bundling from file-tail log events.

4.7.6. ServiceNow incident automation specification

Component	Specification
Script Include: `ResolveApplicationService`	Service-Side: `cmdb_rel_ci` Contains::Contained by from pod CI to Application Service. Spark client: `resolveFromSparkClientLogPath()` when alert text contains `/logs/spark-client/` → Application Service **Spark Client**
Script Include: `K8sLogPodCiBind`	Rebind `em_event` / `em_alert` from initial SGO HOST binding to `cmdb_ci_kubernetes_pod` when log path segment matches a discovered pod `name`; **skip** segment `spark-client`; propagate binding to alerts sharing `message_key`
EM event rule	Mark Spark log events **Ready** when `source=SGO-Dynatrace`, severity matches policy, and `cmdb_ci` is set
EM alert rule	Create alert from Ready events; copy `cmdb_ci`, `description`, `resource`, `node` from event
Business rule: `em-event-bind-k8s-log-pod-ci`	**Before** insert/update on `em_event`; order **5000**; enriches `cmdb_ci`, `resource`, `message_key` via `K8sLogPodCiBind` (must run before EM Ready/alert rules persist the row)

Component	Specification
Business rule: `em-event-propagate-k8s-log-pod-ci`	**After** insert/update on `em_event`; order **5005**; copies pod binding from event to related `em_alert` rows sharing `message_key` (runs after the event row is saved)
Business rule: `em-alert-bind-k8s-log-pod-ci`	**Before** insert/update on `em_alert`; order **5010**; same pod/path enrichment when alert is created or updated directly (covers SGC webhook path that skips a separate bind on the event)
Business rule: `em-alert-create-k8s-log-incident`	**After** insert/update on `em_alert`; order **5020**; tries spark-client path first, else Service-Side Contains; creates or correlates incident, sets **`em_alert.incident`**
Severity filter	Non-prod: `em_alert.severity ⇐ 3` (ERROR and WARN); Production: `em_alert.severity = 2` (ERROR only)
Duplicate suppression	Correlate open incidents by Application Service + short description prefix `Event Log`; use `ProblemID` / `message_key` for event lifecycle

4.7.7. Business rule order and rationale

Business rules on `em_event` and `em_alert` run in **order** within the same **when** phase (`before` or `after`). Lower order runs first.

Order	Rule	Why this position
5000	`em-event-bind-k8s-log-pod-ci` (before)	Parse log path and set `cmdb_ci` / `resource` / `message_key` **before** the row is written. EM event rules that mark the event Ready and copy fields to alerts depend on these values being present on insert.
5005	`em-event-propagate-k8s-log-pod-ci` (after)	After the event is saved, push binding to sibling `em_alert` rows. Must be **after** (not before) because it updates **other** records.
5010	`em-alert-bind-k8s-log-pod-ci` (before)	Same enrichment for alerts created or updated by the SGC webhook / EM alert rule without going through a separate event bind cycle.
5020	`em-alert-create-k8s-log-incident` (before)	Needs enriched `resource` and log-path context from 5000/5010 in the **same** transaction. Creates the incident and sets `em_alert.incident` before the alert row is committed. Using **after** + `current.update()` was unreliable for REST-driven updates.

4.7.8. Alert to incident linkage (`em_alert.incident`)

On **optimizincdemo1**, Event Management links an alert to its incident through **`em_alert.incident`** (reference to the incident record). That field populates the incident **Alerts** tab. Some instances expose the same reference as `em_alert.task`; this build uses **`incident`** **only** — do not reference `task`.

Creating an incident with `GlideRecord('incident').insert()` alone does **not** set this link — automation must assign `current.incident = <incident_sys_id>` on the alert in the **before** business rule.

Incident short description format: **`Event Log WARN`** or **`Event Log ERROR`** (parsed from alert description).

4.7.9. Problem and alert lifecycle (OPEN / RESOLVED)

Spark/Log4j does **not** emit explicit "clear" or "recovery" log lines for this use case. Closure is driven by **Dynatrace problem lifecycle**, not application log content:

1. OneAgent tails application log files on the **host node** (see Log emission).
2. OpenPipeline Davis turns qualifying **ERROR** / **WARN** lines into Davis events; repeated lines merge into one **problem** (`dt.davis.event_timeout: 15m`).
3. When matching log activity stops, Dynatrace **auto-closes** the problem after the timeout.
4. The brooks-lab problem notification has `notifyClosedProblems: true`, so Dynatrace POSTs a **RESOLVED** webhook with the same `ProblemID` as the OPEN post.
5. SGO maps `message_key` = `ProblemID`, so ServiceNow **updates the same** `em_event` (does not create a duplicate).
6. Event Management then transitions the linked **alert** (and optionally the **incident**) according to configured EM rules when the event state/resolution changes.

Verify RESOLVED handling: `em_event` where `source=SGO-Dynatrace` and `resolution_code` is set after problems clear; linked `em_alert.state` should move to **Closed** when EM auto-close rules are configured.

4.7.10. OpenPipeline and custom-source attributes (generic Log4j pattern)

Attributes added by automation (not Spark-specific semantics):

Attribute	Source	Purpose
`spark.log.path`	Custom log source enrichment (`${0}`)	Literal file path of the log line
`spark.application`	Custom log source / OpenPipeline	`spark-client` or pod name (`cmdb_ci_kubernetes_pod.name`)
`spark.client.instance`	Client path segment	Per-driver directory under `spark-client/`
`k8s.pod_name`	Service-side path segment	Must equal `cmdb_ci_kubernetes_pod.name`
`event.name`	OpenPipeline Davis	`Application log {loglevel} on spark-client-{instance}` or `⋯ on {spark.application}`
`event.description`	OpenPipeline Davis	`Application {loglevel} on {spark.log.path}: {content}`

Pod **Kubernetes labels** (`servicenow.io/application-service-identifier`, etc.) are the CMDB/service-mapping contract — not Dynatrace tags. See pod labels.

`resolve-k8s-pod-entity` (OpenPipeline `lookupEntity` for pod Smartscape) is **disabled** so problems stay on **HOST** entities inside the Spark Observability management zone. ServiceNow `K8sLogPodCiBind` rebinds alerts to the **pod CI**

> from the log path; `ResolveApplicationService` traverses the **Contains::Contained by** edge to the Application Service (Service-Side pattern only).

4.7.11. Event Management rules (manual configuration)

Configure under **Event Management → Rules**:

Rule	Condition and action
Event rule (K8s application logs)	**When:** `em_event` insert/update; **Filter:** `source=SGO-Dynatrace` `severity<=3` `cmdb_ci` `ISNOTEMPTY`; **Action:** set state to **Ready** (or use existing Ready rule scoped to SGO-Dynatrace)
Alert rule (K8s application logs)	**When:** Ready event; **Filter:** `source=SGO-Dynatrace`; **Action:** create/update alert; copy `cmdb_ci`, `description`, `resource`, `node`, `message_key` from event

Scope Spark-specific business rules with `source=SGO-Dynatrace` so legacy `source=Dynatrace` events are unaffected during connector migration (see Legacy and SGO-Dynatrace coexistence).

4.7.12. Script Includes

Deploy from `servicenow/integrations/incident/`. Global scope, **Active** = true. Ansible: `ansible/playbooks/servicenow/incident/deploy.yml`.

Each Script Include below summarizes **purpose**, **inputs**, **outputs** / **side effects**, and how it fits the business rules. Source of truth is the `.si.js` file in the repository.

4.7.12.1. ResolveApplicationService

Purpose: Resolve `cmdb_ci_service_discovered` (Application Service) sys_id for incident binding.

Methods:

- `resolveFromInfrastructureCi(ciSysId)` — **Spark Service-Side Log Pattern.** Queries `cmdb_rel_ci` for **Contains::Contained by** where `child` = pod CI and `parent` is an Application Service. Returns parent sys_id or `null`.
- `resolveFromSparkClientLogPath(text)` — **Spark Client-Side Log Pattern.** When `text` contains `/logs/spark-client/`, looks up Application Service **Spark Client** by `identifier=spark-client`, then fallback `name=Spark Client`.

Inputs: Workload CI sys_id (Service-side) or alert description/resource text (Client-side).

Outputs: Application Service sys_id for `incident.cmdb_ci`; no CMDB writes.

Side effects: None (read-only GlideRecord queries).

Source: `servicenow/integrations/incident/ResolveApplicationService.si.js`

4.7.12.2. K8sLogPodCiBind

Purpose: Rebind `em_event` and `em_alert` from HOST or empty CI to `cmdb_ci_kubernetes_pod` when the log path directory segment matches a discovered pod name (OpenPipeline often leaves problems on HOST).

Methods:

- `resolvePodName(description, resource, node)` — Scans concatenated fields for `/…/logs/<segment>/`; skips `spark-client` and wildcards; verifies pod exists in CMDB.
- `applyPodBinding(gr)` — Sets `cmdb_ci`, `node`, `resource`, `message_key` prefix `K8sLog-{pod}-`, and `message` to `Event Log WARN/ERROR` when generic. Returns `true` on success. Caller must `update()`.
- `propagateEventToAlerts(eventGr)` — Copies pod binding from event to related alerts sharing `message_key`.

Inputs: `em_event` or `em_alert` GlideRecord.

Outputs: Mutated fields on current row; optional `em_alert.update()` on related rows.

Source: `servicenow/integrations/incident/K8sLogPodCiBind.si.js`

4.7.13. Business rules (prescriptive configuration)

Create under **System Definition → Business Rules**. All three run **After** insert and update. Filter conditions must include `source=SGO-Dynatrace` so legacy connectors are not modified.

Name	Table	Order	Filter condition
em-event-bind-k8s-log-pod-ci	em_event	5000	source=SGO-Dynatrace
em-alert-bind-k8s-log-pod-ci	em_alert	5010	source=SGO-Dynatrace
em-alert-create-k8s-log-incident	em_alert	5100	source=SGO-Dynatrace^severity⇐3

4.7.13.1. em-event-bind-k8s-log-pod-ci

```
(function executeRule(current, previous /*null on insert*/) {
  var binder = new K8sLogPodCiBind();
  if (!binder.applyPodBinding(current)) {
    return;
  }

  current.update();
  binder.propagateEventToAlerts(current);
})(current, previous);
```

4.7.13.2. em-alert-bind-k8s-log-pod-ci

```
(function executeRule(current, previous /*null on insert*/) {
  var binder = new K8sLogPodCiBind();
  if (!binder.applyPodBinding(current)) {
```

```
    return;
  }

  current.update();
})(current, previous);
```

4.7.13.3. em-alert-create-k8s-log-incident

Creates or correlates an incident on the Application Service, then sets `current.task` so the alert appears on the incident **Alerts** tab.

```
(function executeRule(current, previous /*null on insert*/) {
  if (current.source.toString() !== 'SGO-Dynatrace') {
    return;
  }

  if (!current.task.nil()) {
    return;
  }

  var severity = parseInt(current.severity, 10);
  if (isNaN(severity) || severity > 3) {
    return;
  }

  var desc = current.description.toString();
  if (
    desc.indexOf('Spark ') === -1 &&
    desc.indexOf('spark-app.log') === -1 &&
    desc.indexOf('/mnt/spark/logs/') === -1
  ) {
    return;
  }

  if (current.cmdb_ci.nil()) {
    return;
  }

  var resolver = new ResolveApplicationService();
  var asSysId = resolver.resolveFromInfrastructureCi(current.cmdb_ci.toString());
  if (!asSysId) {
    return;
  }

  var levelMatch = desc.match(/\b(ERROR|WARN)\b/);
  var logLevel = levelMatch
    ? levelMatch[1]
    : severity <= 2
      ? 'ERROR'
      : 'WARN';
  var shortDesc = 'Event Log ' + logLevel;
  var shortDescPrefix = 'Event Log';

  function linkAlertToIncident(incSysId) {
    var alertGr = new GlideRecord('em_alert');
    if (!alertGr.get(current.sys_id)) {
      return;
    }
    alertGr.task = incSysId;
    alertGr.setWorkflow(false);
    alertGr.update();
  }

  var backfillInc = new GlideRecord('incident');
  backfillInc.addQuery('active', true);
```

```
  backfillInc.addQuery('short_description', 'STARTSWITH', shortDescPrefix);
  backfillInc.addEncodedQuery('cmdb_ciISEMPTY');
  backfillInc.orderBy('sys_created_on');
  backfillInc.setLimit(1);
  backfillInc.query();
  if (backfillInc.next()) {
    backfillInc.cmdb_ci = asSysId;
    backfillInc.short_description = shortDesc;
    backfillInc.work_notes =
      'Set Configuration item from alert ' +
      current.number +
      ' (Application Service resolved from pod CI).';
    backfillInc.update();
    linkAlertToIncident(backfillInc.sys_id);
    return;
  }

  var openInc = new GlideRecord('incident');
  openInc.addQuery('cmdb_ci', asSysId);
  openInc.addQuery('active', true);
  openInc.addQuery('short_description', 'STARTSWITH', shortDescPrefix);
  openInc.setLimit(1);
  openInc.query();
  if (openInc.next()) {
    openInc.work_notes =
      'Correlated alert ' +
      current.number +
      ': ' +
      current.description.toString().substring(0, 500);
    openInc.update();
    linkAlertToIncident(openInc.sys_id);
    return;
  }

  var inc = new GlideRecord('incident');
  inc.initialize();
  inc.cmdb_ci = asSysId;
  inc.short_description = shortDesc;
  inc.description = current.description;
  inc.severity = current.severity;
  inc.impact = 2;
  inc.urgency = 2;
  inc.caller_id = gs.getUserID();
  inc.comments = 'Auto-created from Event Management alert ' + current.number;
  inc.work_notes = 'Source alert: ' + current.number;
  var incSysId = inc.insert();
  linkAlertToIncident(incSysId);
})(current, previous);
```

4.7.14. Ansible deployment

```
cd ansible
ansible-playbook -i inventory.yml playbooks/servicenow/incident/deploy.yml \
  -e @../vars/secrets.yaml
```

Playbook path: ansible/playbooks/servicenow/incident/deploy.yml. Source files: servicenow/integrations/incident/*.js.

4.7.15. Example resolution (Kubernetes worker pod)

Phase	Record / relationship
Problem	Davis problem; impacted entity CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890
Event	em_event; cmdb_ci → cmdb_ci_kubernetes_pod myapp-worker-7d4f9b-xk2lm
Alert	em_alert; same pod CI; description contains full ERROR log line
Tag on pod CI	cmdb_key_value: servicenow.io/application-service-identifier=myapp-worker-zone-a
SM edge	Application Service myapp-worker-zone-a Contains pod CI
Incident	incident.cmdb_ci → Application Service myapp-worker-zone-a (via Service-Side Contains traversal)

4.7.16. Application and Dynatrace (Phase 5)

No configuration on the application or Dynatrace side. Dynatrace problem tags (Project, Environment) do not replace CSDM tags on pod CIs for incident binding.

Appendix A: Legacy and SGO-Dynatrace coexistence

During migration from a legacy Event Management push connector (`source=Dynatrace` on the inbound URL) to Service Graph Connector (`source=SGO-Dynatrace`), **both paths may be active**. Use `em_event.source` and `em_alert.source` to scope automation — never assume a single connector.

A.1. Separation by source

Layer	Legacy (`source=Dynatrace`)	SGC (`source=SGO-Dynatrace`)
Inbound URL	`/inbound_event?source=dynatrace&sys_id=<connector-sys-id>`	`/inbound_event?source=SGO-Dynatrace`
CI binding	Legacy connector field mapping + `sys_object_source` (may differ from SGC import)	SGC `ImpactedEntities` → `sys_object_source` (name **SGO-Dynatrace**)
Spark log business rules	**Must not run** — filter `source=SGO-Dynatrace` on every BR	Pod rebind, incident create, Application Service resolution
Problem notification	Pre-existing tenant profiles (for example Demo integrations)	Dedicated profile scoped to integration management zone

A.2. Scoping rules and profiles

- **Dynatrace alerting profiles:** Scope each profile to a **management zone** (or tag filter) so only in-scope problems forward to ServiceNow. A profile attached to a legacy webhook should not receive problems from hosts outside its intended boundary.
- **ServiceNow EM rules:** Add `source=SGO-Dynatrace` (or `source=Dynatrace`) to event and alert rule filter conditions so each rule set applies only to its connector.
- **Business rules:** All Spark log automation in this document filters `source=SGO-Dynatrace`. Legacy CPU or infrastructure alerts continue through legacy rules until that connector is retired.

A.3. Host CPU alerts (not Spark-specific)

Metric events on `builtin:host.cpu.usage` fire per **host**, not per application. Even on the SGC path, a CPU problem binds `em_alert.cmdb_ci` to the breaching **host CI** — that is correct behavior.

To avoid misleading titles and out-of-scope hosts:

1. Use a **neutral title**: `Host ({dims:dt.entity.host}) CPU above 80%` (not application-branded text).
2. Add an **entity filter** on the metric event: **management zone** (preferred) or explicit **HOST_NAME** list for integration hosts.

3. Create **separate metric events** per organization or environment when thresholds differ (for example 80% in lab, 90% in production).

Template: `observability/dynatrace/tenants/<tenant>/integrations/spark-cpu-metric-event.json.j2`. Deploy via `ansible/playbooks/servicenow/sgc/sources/dynatrace/tasks/apply_dt_anomaly_detectors.yml`.

A.4. When to retire legacy connector

After SGC cutover is verified (events arrive with `source=SGO-Dynatrace`, pod CI binding works, incidents link on **Alerts** tab):

1. Disable legacy Dynatrace problem notification profiles pointing at `source=dynatrace`.
2. Leave legacy EM rules in place until no new `source=Dynatrace` events arrive.
3. Remove legacy rules once the connector is decommissioned.